Students:
Nguyen Huu Truong Giang [u6110634@udg.edu]

Lab 6: Task-Priority kinematic control (2B)

1 Introduction

In a mobile manipulator, the robot can accomplish a desired end-effector task through the Jacobian matrix, which provides both the mobile base velocity and the robot arm joint velocities. With this type of robot, we can also incorporate additional inequality constraints, such as joint limits, while still enabling the robot to complete the end-effector task.

In this lab, we introduce mobile manipulator kinematic control to achieve the desired end-effector task. Moreover, we combine the weighted Damped Least Squares method with mobile manipulator kinematic control to evaluate the robot's behavior under different weight values. We also implement three different mobile base kinematic strategies—move-then-rotate, rotate-then-move, and simultaneous movement and rotation—to observe the differences in both the mobile base trajectory and the end-effector path under these control approaches.

Section 2 presents the equations of mobile manipulator kinematics, the weighted Damped Least Squares method, and the three different mobile base kinematic strategies. Section 3 illustrates the experimental results under different scenarios and discusses the key observations. Finally, Section 4 concludes the lab.

2 Methodology

The implementation was carried out in Python using the `NumPy` library for numerical computations and `Matplotlib` for visualization of the robot motion and results. The mobile manipulator kinematics, the weighted Damped Least Squares method, and the different mobile base kinematic strategies are described below.

2.1 Mobile Manipulator Kinematics

The kinematics of a mobile manipulator are described by the following equation:

$$ {}^{I}T_E = {}^{I}T_B \, {}^{B}T_0 \prod_{i=1}^{n} {}^{i-1}T_i \tag{1} $$

where ${}^{I}T_B$ represents the mobile base transformation, defined as:

$$ {}^{I}T_B = \begin{bmatrix} {}^{I}R_B & {}^{I}\eta_1 \\ 0_{1\times 3} & 1 \end{bmatrix} \tag{2} $$

The rotation matrix ${}^{I}R_B$ is given by:

$$ {}^{I}R_B = R_z(\psi)R_y(\theta)R_x(\phi) \tag{3} $$

which can be expanded as:

$$ {}^{I}R_B = \begin{bmatrix} \cos\psi\cos\theta & \sin\psi\cos\theta & -\sin\theta \\ -\sin\psi\cos\phi + \cos\psi\sin\theta\sin\phi & \cos\psi\cos\phi + \sin\psi\sin\theta\sin\phi & \cos\theta\sin\phi \\ \sin\psi\sin\phi + \cos\psi\sin\theta\cos\phi & -\cos\psi\sin\phi + \sin\psi\sin\theta\cos\phi & \cos\theta\cos\phi \end{bmatrix} \tag{4} $$

The term ${}^{B}T_0$ denotes the manipulator base transformation:

$$ {}^{B}T_0 = \begin{bmatrix} {}^{B}R_0 & {}^{B}\eta_{1,0} \\ 0_{1\times 3} & 1 \end{bmatrix} \tag{5} $$

The term ${}^{i-1}T_i$ represents the Denavit–Hartenberg (DH) transformation between consecutive manipulator frames:

$$ {}^{i-1}T_i = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos q_i & -\sin q_i & 0 & 0 \\ \sin q_i & \cos q_i & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & a_i \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\alpha_i & -\sin\alpha_i & 0 \\ 0 & \sin\alpha_i & \cos\alpha_i & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{6} $$

2.2 Weighted Damped Least Squares

The weighted Damped Least Squares (DLS) method is described by the following equations:

$$ \zeta = J^T(\mathbf{q}) \left(J(\mathbf{q})J^T(\mathbf{q}) + \lambda^2 I \right)^{-1} \dot{\mathbf{x}}_E \tag{7} $$

$$ \zeta = W^{-1}J^T(\mathbf{q}) \left(J(\mathbf{q})W^{-1}J^T(\mathbf{q}) + \lambda^2 I \right)^{-1} \dot{\mathbf{x}}_E \tag{8} $$

where W is the weighting matrix defined as:

$$ W \in \mathbb{R}^{n\times n}, \quad W = \mathrm{diag}(w_1, w_2, \ldots, w_n) \tag{9} $$

Moreover, a higher weight reduces the contribution of the corresponding degree of freedom (DOF) in the solution.

2.3 Mobile Base Kinematic Strategies

Three different mobile base kinematic strategies are used to control the mobile base, as described below.

2.3.1 Move then rotate

The robot first translates and then rotates. The position and orientation are updated as follows:

$$ x_t = x_{t-1} + v\cos(\theta_{t-1})\,\Delta t \tag{10} $$

$$ y_t = y_{t-1} + v\sin(\theta_{t-1})\,\Delta t \tag{11} $$

$$ \theta_t = \theta_{t-1} + \omega\,\Delta t \tag{12} $$

2.3.2 Rotate then move

The robot first rotates and then translates. The update equations are:

$$\theta_t = \theta_{t-1} + \omega\,\Delta t \tag{13}$$
$$x_t = x_{t-1} + v\cos(\theta_t)\,\Delta t \tag{14}$$
$$y_t = y_{t-1} + v\sin(\theta_t)\,\Delta t \tag{15}$$

2.3.3 Simultaneous rotation and translation

The robot rotates and translates simultaneously. The motion is described by:

$$x_{t+1} = \begin{cases} x_t + v\cos(\theta_t)\,\Delta t, & \text{if } |\omega| << 0 \\ x_t + \dfrac{v}{\omega}\left[\sin(\theta_t + \omega\Delta t) - \sin(\theta_t)\right], & \text{otherwise} \end{cases} \tag{16}$$

$$y_{t+1} = \begin{cases} y_t + v\sin(\theta_t)\,\Delta t, & \text{if } |\omega| << 0 \\ y_t + \dfrac{v}{\omega}\left[-\cos(\theta_t + \omega\Delta t) + \cos(\theta_t)\right], & \text{otherwise} \end{cases} \tag{17}$$

$$\theta_{t+1} = \theta_t + \omega\Delta t \tag{18}$$

where x and y denote the robot position, θ denotes the robot orientation, v is the linear velocity in the robot frame, ω is the angular velocity, and Δt is the sampling time.

3 Results

Three main experiments were conducted in this study. In the first experiment, mobile manipulator kinematic control was implemented with a task-priority framework, where the end-effector desired position was defined as the primary task and joint limit avoidance was considered as an inequality constraint.

In the second experiment, the Damped Least Squares method was replaced with the Weighted Damped Least Squares method within the same task-priority framework. Three different weight values were tested to evaluate their effect on the behavior of the mobile manipulator.

Finally, three different mobile base kinematic strategies were formulated and implemented: move-then-rotate, rotate-then-move, and simultaneous rotation and translation. These strategies were compared to analyze their effects on both the mobile base trajectory and the end-effector path.

In this laboratory experiment, a mobile base with a 3-DOF planar manipulator consisting of three revolute joints was used. The robot model, including its coordinate frames and Denavit–Hartenberg (DH) parameters, is illustrated in Fig. 1.

The robot configuration is initialized according to the DH parameters listed in Table 1.

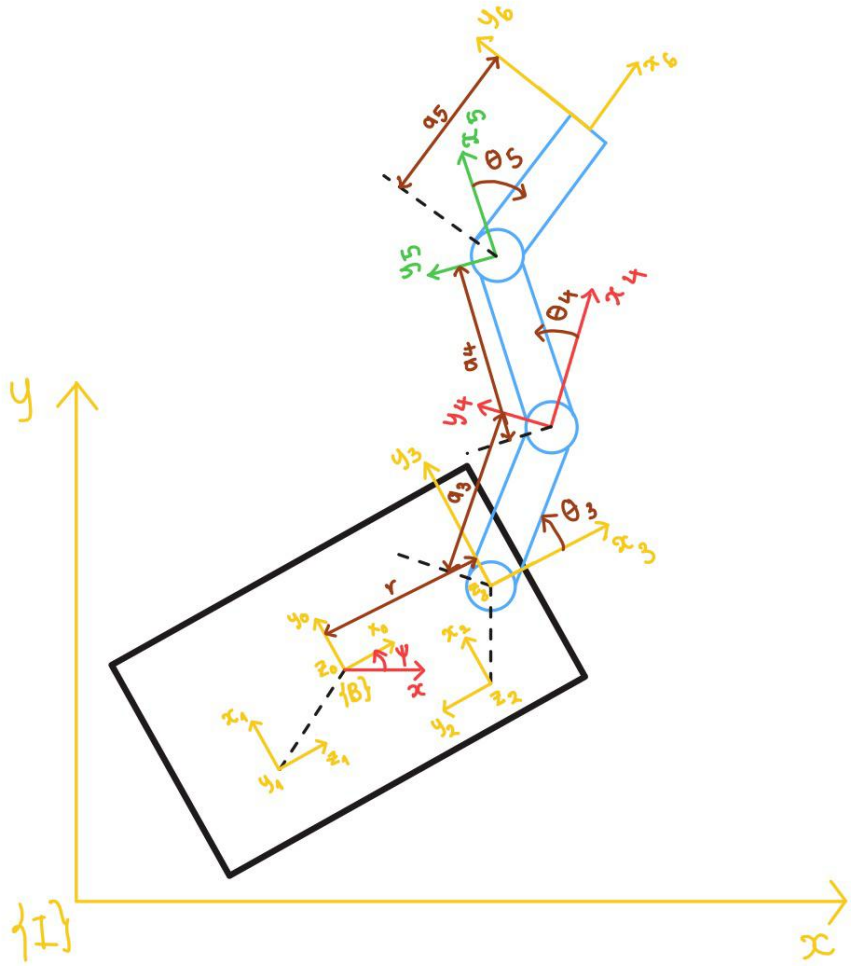


Figure 1: Visualization of the mobile manipulator

DOF	θ_i	d_i	a_i	α_i	Home
1	$\pi/2$	0	0	$\pi/2$	0
2	0	$r = 0.15$	0	$-\pi/2$	0
3	q_1	0	$a_1 = 0.4$	0	0
4	q_2	0	$a_2 = 0.3$	0	0
5	q_3	0	$a_3 = 0.2$	0	0

Table 1: Denavit–Hartenberg parameters of the mobile manipulator with 3 DOF robot arm

3.1 Mobile manipulator kinematic with joint limit

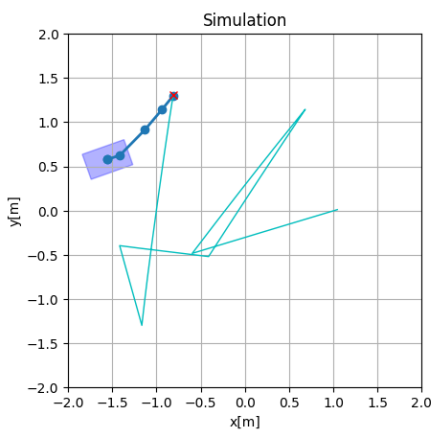
In this experiment, we implement mobile manipulator kinematic to control robot move end-effector of robot arm desired point with enequality task joint limit.

Before conducting the experiment, several parameters are defined and summarized in Table 2.

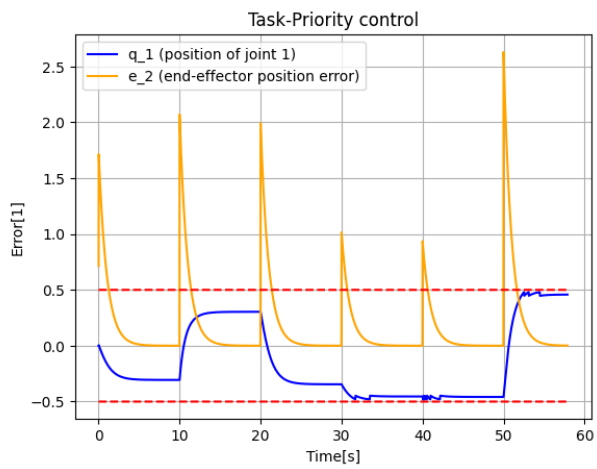
Joint 1 limit	$[-0.5, 0.5]$
Deactivation threshold (δ)	0.05
Activation thershold (α)	0.02
Total simulation time	10
Damping factor	0.1

Table 2: Parameters for the joint limit experiment

After setting these parameters, the experiments were conducted, and the results are shown in Fig. 2a and Fig. 2b. The complete Python code implementation is provided in Appendices A.1, A.2 and A.3.



(a) Simulation of the mobile manipulator, including end-effector goal.



(b) Evolution of the

Figure 2: Evolution of the TP control error and first manipulator joint position, over time.

As shown in Fig. 2b, the robot end-effector successfully reaches the desired point, with the position error converging to zero approximately every 10 seconds. Additionally, the position of joint 1 remains within the range of -0.5 to 0.5.

Some oscillations can be observed in the position of joint 1, as it operates near the task-switching boundary. However, as illustrated in Fig. 2a, the end-effector trajectory remains smooth despite these oscillations.

Compared to the previous lab, the key difference is that the robot base is mobile in this case. As a result, the end-effector is able to reach any desired position even in the presence of inequality constraints such as joint limits.

3.2 Weight Damping Least Square

In this experiment, we implement weight damping least square with task priority end-effector configuration with three different weight values to see the different behavior mobile manipulator. Before conduct the experiment, several parameters have to be defined and summarized in Table 3.

Weight value 1	$[1.0, 1.0, 10.0, 0.5, 0.5]$
Weight value 2	$[1.0, 1.0, 10.0, 10.0, 10.0]$
Weight value 3	$[10.0, 10.0, 1.0, 1.0, 1.0]$
Total simulation time	10
Damping factor	0.1

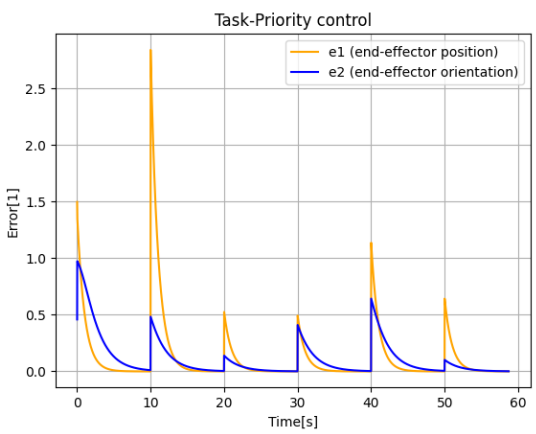
Table 3: Parameters for the weight Damping Least Square experiment

3.2.1 Case 1: Restricting first joint of manipulation

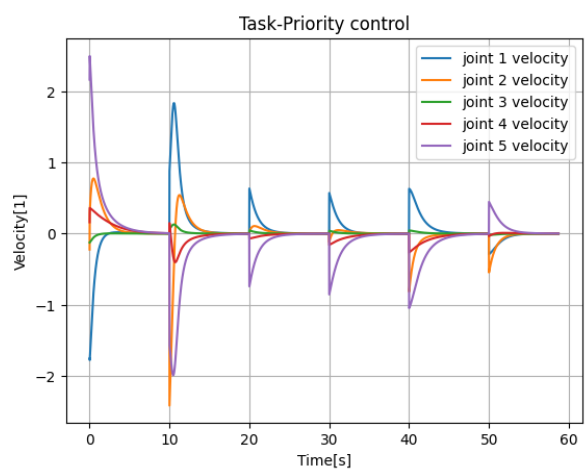
As shown in Fig. 3a, the mobile manipulator successfully moves the end-effector to the desired configuration. However, in this case, the weight value of the first joint is increased, which reduces its level of activity.

As illustrated in Fig. 3b, joint 3 becomes significantly less active compared to the other joints, remaining close to zero over time. In contrast, the weights of joints 4 and 5 are decreased, which increases their activity. As a result, the velocities of joints 4 and 5 frequently deviate from zero and can sometimes reach relatively high values.

This indicates that the robot tends to rely more on joints 4 and 5, along with the mobile base, to accomplish the task, while reducing the use of the first manipulator joint.



(a) The evolution of end-effector configuration task error over time for case 1.



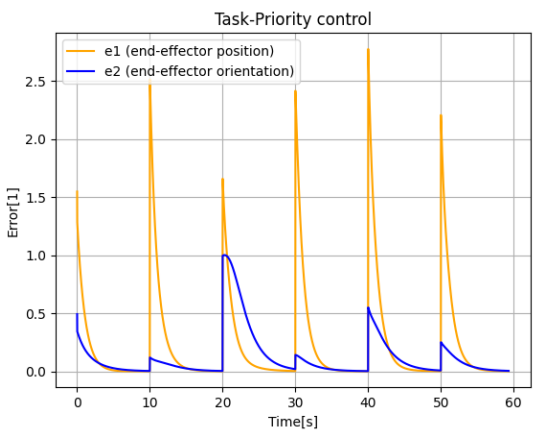
(b) The evolution of the velocity output from TP algorithm, for all DOF, over time for case 1.

Figure 3: Result for Weight DLS experiment with case 1

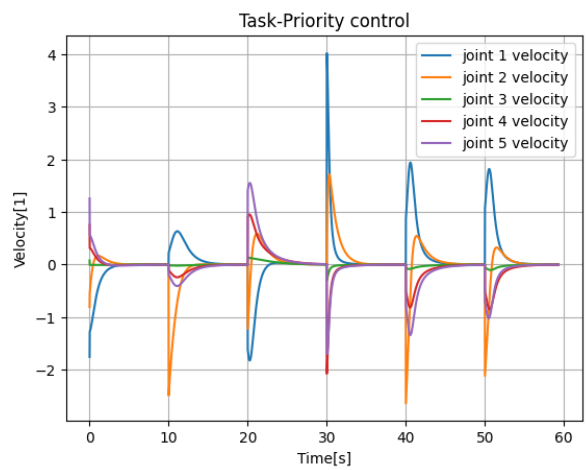
3.2.2 Case 2: Restricting all three manipulator joints

As shown in Fig. 4a, the mobile manipulator successfully moves the end-effector to the desired configuration. In this case, the weights of all three manipulator joints are increased, which reduces their activity.

Therefore, as illustrated in Fig. 4b, joints 3, 4, and 5 are significantly less active than the other joints. The velocities of these three manipulator joints are much lower than those of the first two degrees of freedom of the mobile base for most of the time, which indicates that the robot tends to rely more on the mobile base to accomplish the task while minimizing the activity of the three manipulator joints. Nevertheless, the mobile manipulator is still able to achieve the desired end-effector configuration.



(a) The evolution of end-effector configuration task error over time for case 2.



(b) The evolution of the velocity output from TP algorithm, for all DOF, over time for case 2.

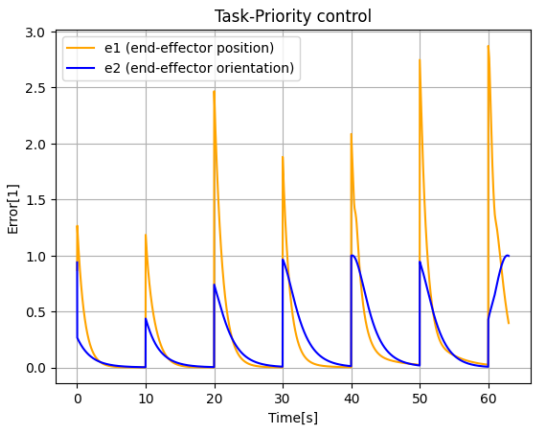
Figure 4: Result for Weight DLS experiment with case 2

3.2.3 Case 3: Restricting the first two mobile base joint velocities

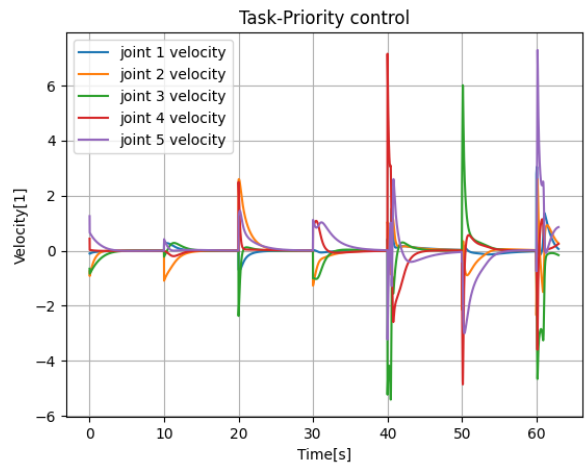
As shown in Fig. 5a, the mobile manipulator successfully moves the end-effector to the desired configuration. In this case, the weights of the two mobile base joints are increased, which reduces their activity.

Therefore, as illustrated in Fig. 5b, joints 1 and 2 are significantly less active than the other joints. The velocities of the mobile base are much lower than those of the three manipulator joints. This indicates that the robot tends to rely more on the manipulator to accomplish the task while minimizing the use of the mobile base.

Nevertheless, the mobile manipulator is still able to achieve the desired end-effector configuration.



(a) The evolution of end-effector configuration task error over time for case 3.



(b) The evolution of the velocity output from TP algorithm, for all DOF, over time for case 3.

Figure 5: Result for Weight DLS experiment with case 3

The complete python implementation is presented in the Appendices A.1, A.2 and A.4.

3.3 Three different mobile base kinematic strategies

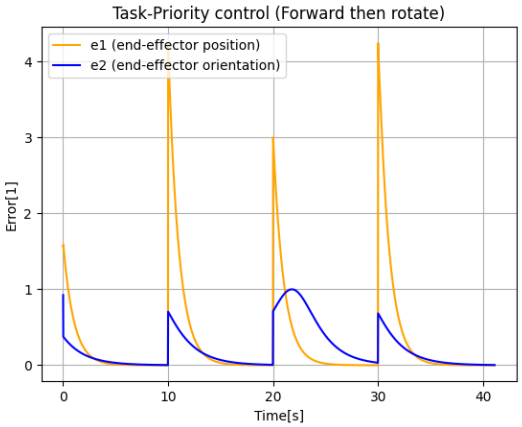
In this experiment, three different mobile base kinematic strategies were implemented to follow a pre-defined trajectory. The behavior of the robot was then analyzed by comparing both the mobile base and end-effector paths.

To conduct the experiment, several key parameters were first defined and are summarized in Table 4.

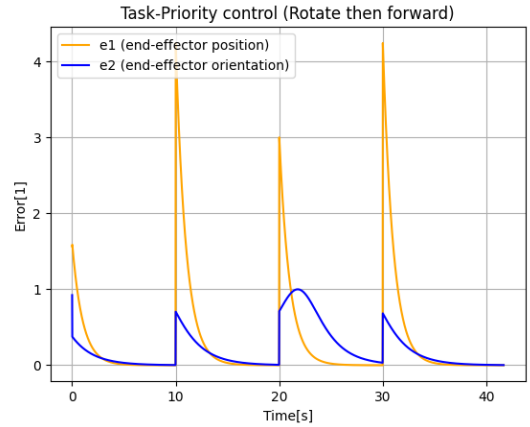
Way point 1	$[1.5, 1.5, \pi/4]$
Way point 2	$[-1.5, -1.5, -\pi/4]$
Way point 3	$[1.5, -1.5, -3\pi/4]$
Way point 4	$[-1.5, -1.5, 3\pi/4]$
Total simulation time	10
Damping factor	0.1

Table 4: Parameters for the mobile base kinematic strategies

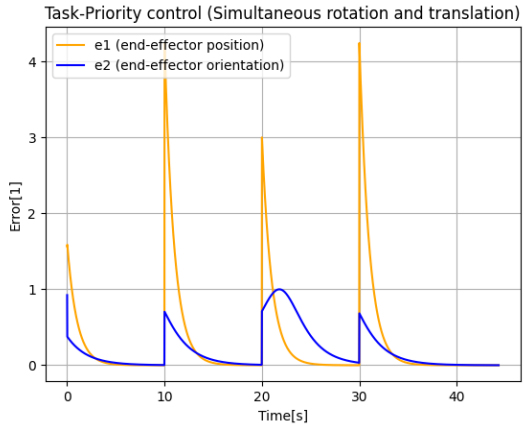
As shown in Fig. 6, all three mobile base kinematic strategies are able to control the robot to move the end-effector to the desired configuration. The error in both position and orientation converges to zero approximately every 10 seconds.



(a) End-effector configuration error over time (forward-rotate motion).

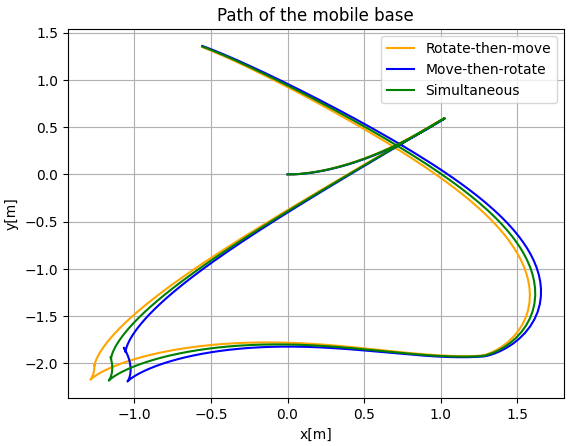


(b) End-effector configuration error over time (rotate-forward motion).

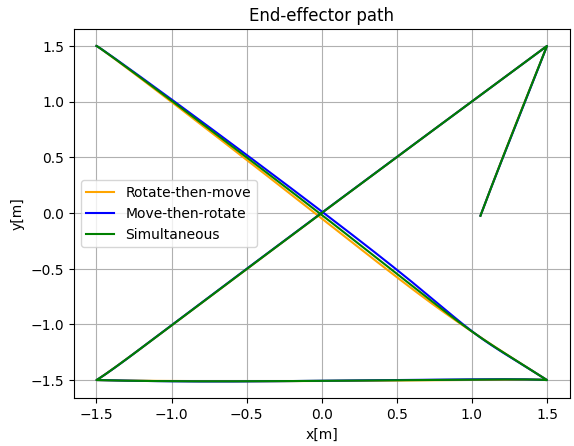


(c) End-effector configuration error over time (simultaneous motion).

Figure 6: End-effector configuration error over time in three mobile base kinematic strategies.



(a) The visualization of mobile base path.



(b) The visualization of end-effector path.

Figure 7: The visualization path of mobile base and end-effector of manipulation

Moreover, as illustrated in Fig. 7a and Fig. 7b, there are noticeable differences in both the mobile base trajectories and the end-effector paths among the three kinematic strategies. The differences in the end-effector paths are relatively small; in particular, the trajectory corresponding to simultaneous rotation and translation lies between the move-then-rotate and rotate-then-move strategies.

These differences are more pronounced in the mobile base trajectories, especially along curved paths. Simultaneous motion produces a path "in between" because it continuously distributes motion between translation and rotation, rather than doing one completely before the other. The trajectory naturally lies geometrically between the extreme cases of move-then-rotate and rotate-then-move.

The complete python implementation is presented in the Appendices A.1, A.2, A.5, and A.6.

4 Conclusion

In conclusion, this laboratory exercise demonstrates the implementation of a mobile manipulator with task-priority control and inequality constraints. Through simulation and analysis of the mobile manipulator's trajectory, end-effector path, and the corresponding errors in end-effector and joint positions, we gain a clear understanding of how the mobile manipulator operates to achieve its tasks.

The experiment highlights the advantages of incorporating a mobile base when inequality constraints are present. Compared to a fixed manipulator with joint limits, the mobile manipulator can complete tasks more easily by leveraging base mobility. Furthermore, the use of the Weighted

Damped Least Squares method provides insight into how different weightings influence the distribution of motion among the manipulator and mobile base joints.

Finally, the comparison of three mobile base kinematic strategies shows that, while the choice of strategy slightly affects the paths of the mobile base and end-effector, it does not prevent the manipulator from successfully completing the desired task.

A Appendix: Code Implementations

This appendix contains all Python code implementations used in the laboratory exercise. To improve clarity and readability of the main report, all figures presenting source code are moved to this appendix.

A.1 lab2_robotics.py

```python
import numpy as np  # Import Numpy
from math import cos, sin


def DH(d, theta, a, alpha):
    """
    Function builds elementary Denavit-Hartenberg transformation matrices
    and returns the transformation matrix resulting from their multiplication.

    Arguments:
    d (double): displacement along Z-axis
    theta (double): rotation around Z-axis
    a (double): displacement along X-axis
    alpha (double): rotation around X-axis

    Returns:
    (Numpy array): composition of elementary DH transformations
    """
    # 1. Build matrices representing elementary transformations (based on input parameters).
    T_d = np.array([[1, 0, 0, 0],
                    [0, 1, 0, 0],
                    [0, 0, 1, d],
                    [0, 0, 0, 1]])
    T_theta = np.array(
        [
            [cos(theta), -sin(theta), 0, 0],
            [sin(theta), cos(theta), 0, 0],
            [0, 0, 1, 0],
            [0, 0, 0, 1],
        ]
    )
    T_a = np.array([[1, 0, 0, a],
                    [0, 1, 0, 0],
                    [0, 0, 1, 0],
                    [0, 0, 0, 1]])
    T_alpha = np.array(
        [
            [1, 0, 0, 0],
            [0, cos(alpha), -sin(alpha), 0],
            [0, sin(alpha), cos(alpha), 0],
            [0, 0, 0, 1],
```

```
        ]
    )
    # 2. Multiply matrices in the correct order (result in T).
    T = T_d @ T_theta @ T_a @ T_alpha
    return T


def kinematics(d, theta, a, alpha, Tb=np.eye(4)):
    """
    Functions builds a list of transformation matrices, for a kinematic chain,
    descried by a given set of Denavit-Hartenberg parameters.
    All transformations are computed from the base frame.

    Arguments:
    d (Numpy array): list of displacements along Z-axis
    theta (Numpy array): list of rotations around Z-axis
    a (Numpy array): list of displacements along X-axis
    alpha (Numpy array): list of rotations around X-axis

    Returns:
    (list of Numpy array): list of transformations along the kinematic chain (from the base frame)
    """
    T = [Tb]  # Base transformation
    # For each set of DH parameters:
    for i in range(len(d)):
        # 1. Compute the DH transformation matrix.
        n_1_T_n = DH(d[i], theta[i], a[i], alpha[i])
        # 2. Compute the resulting accumulated transformation from the base frame.
        T_0_n = T[-1] @ n_1_T_n
        # Rotate to match the transformation from the base frame to the first joint of robot arm ( Rotation of -90 degrees around Z-axis)
        if i == 1:
            T_0_n = T_0_n @ np.array([[0, 1, 0, 0],
                                      [-1, 0, 0, 0],
                                      [0, 0, 1, 0],
                                      [0, 0, 0, 1]])
        # 3. Append the computed transformation to T.
        T.append(T_0_n)
    return T


# Inverse kinematics
def jacobian(T, revolute):
    """
    Function builds a Jacobian for the end-effector of a robot,
    described by a list of kinematic transformations and a list of joint types.

    Arguments:
```

```
    T (list of Numpy array): list of transformations along the kinematic chain of the robot (from the base frame)
    revolute (list of Bool): list of flags specifying if the corresponding joint is a revolute joint

    Returns:
    (Numpy array): end-effector Jacobian
    """
    # 1. Initialize J and O.
    n_dof = len(revolute)
    J = np.zeros((6, n_dof))
    O = T[-1][0:3, 3]
    # 2. For each joint of the robot
    for i in range(n_dof):
        #   a. Extract z and o.
        i_1_T_i = T[i]
        z = i_1_T_i[0:3, 2]
        o = i_1_T_i[0:3, 3]
        #   b. Check joint type.
        if revolute[i] == True:
            cross_vector = np.array([np.cross(z, (O - o))]).T
            z = np.array([z]).T
            J_i = np.vstack((cross_vector, z))
        else:
            J_i = np.vstack((z, np.zeros((3, 1))))
        #   c. Modify corresponding column of J.
        J[:, i] = J_i[:, 0]
    return J


# Damped Least-Squares
def DLS(A, damping):
    """
    Function computes the damped least-squares (DLS) solution to the matrix inverse problem.

    Arguments:
    A (Numpy array): matrix to be inverted
    damping (double): damping factor

    Returns:
    (Numpy array): inversion of the input matrix
    """
    DLS_matrix = A.T @ np.linalg.inv(A @ A.T + damping**2 * np.identity(len(A)))
    return DLS_matrix  # Implement the formula to compute the DLS of matrix A.

# Damped Least-Squares
def weighted_DLS(A, damping, W):
    """
```

```
    Function computes the damped least-squares (DLS) solution to the matrix inverse problem.

    Arguments:
    A (Numpy array): matrix to be inverted
    damping (double): damping factor
    weight (Numpy array): weight matrix

    Returns:
    (Numpy array): inversion of the input matrix
    """
    DLS_weighted_matrix = np.linalg.inv(W) @ A.T @ np.linalg.inv(A @ np.linalg.inv(W) @ A.T + damping**2 * np.identity(len(A)))
    return DLS_weighted_matrix  # Implement the formula to compute the DLS of matrix A.


# Extract characteristic points of a robot projected on X-Y plane
def robotPoints2D(T):
    """
    Function extracts the characteristic points of a kinematic chain on a 2D plane,
    based on the list of transformations that describe it.

    Arguments:
    T (list of Numpy array): list of transformations along the kinematic chain of the robot (from the base frame)

    Returns:
    (Numpy array): an array of 2D points
    """
    P = np.zeros((2, len(T)))
    for i in range(len(T)):
        P[:, i] = T[i][0:2, 3]
    return P
```

A.2 lab6_robotics.py

```
from lab4_robotics import *

class MobileManipulator:
    '''
        Constructor.

        Arguments:
        d (Numpy array): list of displacements along Z-axis
        theta (Numpy array): list of rotations around Z-axis
        a (Numpy array): list of displacements along X-axis
        alpha (Numpy array): list of rotations around X-axis
        revolute (list of Bool): list of flags specifying if the corresponding joint is a revolute joint
    '''
    def __init__(self, d, theta, a, alpha, revolute):
        self.d = d
        self.theta = theta
        self.a = a
        self.alpha = alpha
        self.revolute = revolute
        self.revoluteExt = np.concatenate([np.array([True, False]), self.revolute])        # List of joint types extended with base joints
        self.r = 0.15                                                                       # Distance from robot centre to manipulator base
        self.dof = len(self.revoluteExt)                                                    # Number of DOF of the system
        self.q = np.zeros((len(self.revolute),1))                                           # Vector of joint positions (manipulator)
        self.eta = np.zeros((3,1))                                                          # Vector of base pose (position & orientation)
        self.update(np.zeros((self.dof,1)), 0.0)                                            # Initialise robot state

    '''
        Method that updates the state of the robot.

        Arguments:
        dQ (Numpy array): a column vector of quasi velocities
        dt (double): sampling time
    '''
    def update(self, dQ, dt):
        # Update manipulator
        self.q += dQ[2:, 0].reshape(-1,1) * dt
        for i in range(len(self.revolute)):
```

```
            if self.revolute[i]:
                self.theta[i] = self.q[i][0]
            else:
                self.d[i] = self.q[i][0]

        # Update mobile base pose

        # First move forward. then rotate.
        # self.eta[0][0] += dQ[1][0] * np.cos(self.eta[2][0]) * dt
        # self.eta[1][0] += dQ[1][0] * np.sin(self.eta[2][0]) * dt
        # self.eta[2][0] += dQ[0][0] * dt

        # First rotate, then move forward.
        # self.eta[2][0] += dQ[0][0] * dt
        # self.eta[0][0] += dQ[1][0] * np.cos(self.eta[2][0]) * dt
        # self.eta[1][0] += dQ[1][0] * np.sin(self.eta[2][0]) * dt

        # Simultaneous rotation and translation
        if np.abs(dQ[0][0]) < 1e-6:
            self.eta[0][0] += dQ[1][0] * np.cos(self.eta[2][0]) * dt
            self.eta[1][0] += dQ[1][0] * np.sin(self.eta[2][0]) * dt
        else:
            self.eta[0][0] += (dQ[1][0] / dQ[0][0]) * (np.sin(self.eta[2][0] + dQ[0][0] * dt) - np.sin(self.eta[2][0]))
            self.eta[1][0] += (dQ[1][0] / dQ[0][0]) * (-np.cos(self.eta[2][0] + dQ[0][0] * dt) + np.cos(self.eta[2][0]))

        self.eta[2][0] += dQ[0][0] * dt


        # Base kinematics
        R_b = np.array([[cos(self.eta[2][0]), -sin(self.eta[2][0]), 0],
                        [sin(self.eta[2][0]), cos(self.eta[2][0]), 0],
                        [0, 0, 1]])
        Tb = np.array([[R_b[0, 0], R_b[0, 1], R_b[0, 2], self.eta[0][0]],
                       [R_b[1, 0], R_b[1, 1], R_b[1, 2], self.eta[1][0]],
                       [R_b[2, 0], R_b[2, 1], R_b[2, 2], 0],
                       [0, 0, 0, 1]])           # Transformation of the mobile base



        # Combined system kinematics (DH parameters extended with base DOF)
        dExt = np.concatenate([np.array([0, self.r]), self.d])
        thetaExt = np.concatenate([np.array([np.pi/2, 0]), self.theta])
        aExt = np.concatenate([np.array([0, 0]), self.a])
        alphaExt = np.concatenate([np.array([np.pi/2, -np.pi/2]), self.alpha])

        self.T = kinematics(dExt, thetaExt, aExt, alphaExt, Tb)

    '''
```

```
        Method that returns the characteristic points of the robot.
    '''
    def drawing(self):
        return robotPoints2D(self.T)

    '''
        Method that returns the end-effector Jacobian.
    '''
    def getEEJacobian(self):
        return jacobian(self.T, self.revoluteExt)

    '''
        Method that returns the end-effector transformation.
    '''
    def getEETransform(self):
        return self.T[-1]

    '''
        Method that returns the position of a selected joint.

        Argument:
        joint (integer): index of the joint

        Returns:
        (double): position of the joint
    '''
    def getJointPos(self, joint):
        return self.q[joint-2]


    def getBasePose(self):
        return self.eta

    '''
        Method that returns number of DOF of the manipulator.
    '''
    def getDOF(self):
        return self.dof

    ###
    def getLinkJacobian(self, link):
        return jacobianLink(self.T, self.revoluteExt, link)

    def getLinkTransform(self, link):
        return self.T[link]
```

A.3 Full code for mobile manipulator with joint limit task

```
from lab6_robotics import * # Includes numpy import
import matplotlib.pyplot as plt
import matplotlib.patches as patch
import matplotlib.animation as anim
import matplotlib.transforms as trans

# Robot model
d = np.zeros(3)                              # displacement along Z-axis
theta = np.array([0.2, 0.5, 0.2])            # rotation around Z-axis
alpha = np.zeros(3)                          # rotation around X-axis
a = np.array([0.4, 0.3, 0.2])                # displacement along X-axis
revolute = np.array([True, True, True])      # flags specifying the type of joints
robot = MobileManipulator(d, theta, a, alpha, revolute) # Create the robot model
damping_factor = 0.1

# Task definition
joint_limit_1 = np.array([-0.5, 0.5])

tasks = [
          JointLimit("Joint Limit", joint_limit_1, 2),
          Position2D("End-effector position", np.array([-0.5, 1.0]).reshape(2,1), 5)
        ]

# Record the data for plotting
error_end_effector = []
joint_1_position = []

# Simulation params
dt = 1.0/60.0

# Drawing preparation
fig = plt.figure()
ax = fig.add_subplot(111, autoscale_on=False, xlim=(-2, 2), ylim=(-2,2))
ax.set_title('Simulation')
ax.set_aspect('equal')
ax.grid()
ax.set_xlabel('x[m]')
ax.set_ylabel('y[m]')
rectangle = patch.Rectangle((-0.25, -0.15), 0.5, 0.3, color='blue', alpha=0.3)
veh = ax.add_patch(rectangle)
line, = ax.plot([], [], 'o-', lw=2) # Robot structure
path, = ax.plot([], [], 'c-', lw=1) # End-effector path
point, = ax.plot([], [], 'rx') # Target
PPx = []
PPy = []

```

```python
# Simulation initialization
def init():
    global tasks
    line.set_data([], [])
    path.set_data([], [])
    point.set_data([], [])
    tasks[1].setDesired((np.random.rand(2,1)*3-1.5)) # Random target
    return line, path, point

# Simulation loop
def simulate(t):
    global tasks
    global robot
    global PPx, PPy

    ### Recursive Task-Priority algorithm
    # Initialize null-space
    P = np.identity(robot.dof)
    # Initialize output vector (joint velocity)
    dq = np.zeros((robot.dof, 1))
    # Loop over tasks
    for i in range(len(tasks)):

        tasks[i].update(robot)

        if tasks[i].isActivate() != 0:
            # Update task state
            # Padding Jacobian to match the dimension of the robot dof and compute the augmented Jacobian, then compute the DLS solution and accumulate velocity, and finally update the null-space projector
            Jacobian_padded = np.pad(tasks[i].getJacobian(), ((0, 0), (0, robot.dof - tasks[i].getJacobian().shape[1])))
            # Compute augmented Jacobian
            J_bar = Jacobian_padded @ P
            # Task definition
            x_dot = tasks[i].getFeedforwardVelocity() + tasks[i].getGainMatrixK() @ tasks[i].getError()
            # Compute task velocity & Accumulate velocity
            dq = dq + DLS(J_bar, damping_factor) @ (tasks[i].a * x_dot - Jacobian_padded @ dq)
            # Update null-space projector
            P = P - np.linalg.pinv(J_bar) @ J_bar
        else:
            dq = dq
            P = P
    ###

    # Record data for plotting
    error_end_effector.append(np.linalg.norm(tasks[1].getError()))
    joint_1_position.append(robot.getJointPos(2)[0])
```

```python

    # Update robot
    robot.update(dq, dt)

    # Update drawing
    # -- Manipulator links
    PP = robot.drawing()
    line.set_data(PP[0,:], PP[1,:])
    PPx.append(PP[0,-1])
    PPy.append(PP[1,-1])
    path.set_data(PPx, PPy)
    point.set_data(tasks[1].getDesired()[0], tasks[1].getDesired()[1])
    # -- Mobile base
    eta = robot.getBasePose()
    veh.set_transform(trans.Affine2D().rotate(eta[2,0]) + trans.Affine2D().translate(eta[0,0], eta[1,0]) + ax.transData)

    return line, veh, path, point

# Run simulation
animation = anim.FuncAnimation(fig, simulate, np.arange(0, 10, dt),
                                interval=10, blit=True, init_func=init, repeat=True)
plt.show()

# Plotting the end-effector path and distance to the obstacles
tvec = np.arange(0, len(error_end_effector)*dt, dt)
plt.figure()
plt.plot(tvec, joint_1_position, label='q_1 (position of joint 1)', color='blue')
plt.plot(tvec, error_end_effector, label='e_2 (end-effector position error)', color='orange')
plt.plot(tvec, np.ones_like(tvec)*joint_limit_1[0], color='red', linestyle='--')
plt.plot(tvec, np.ones_like(tvec)*joint_limit_1[1], color='red', linestyle='--')
plt.xlabel('Time[s]')
plt.ylabel('Error[1]')
plt.title('Task-Priority control')
plt.legend()
plt.grid()
plt.show()
```

A.4 Full code for weight Damping Least Square

```
from lab6_robotics import * # Includes numpy import
import matplotlib.pyplot as plt
import matplotlib.patches as patch
import matplotlib.animation as anim
import matplotlib.transforms as trans

# Robot model
d = np.zeros(3)                              # displacement along Z-axis
theta = np.array([0.2, 0.5, 0.2])            # rotation around Z-axis
alpha = np.zeros(3)                          # rotation around X-axis
a = np.array([0.4, 0.3, 0.2])                # displacement along X-axis
revolute = np.array([True, True, True])      # flags specifying the type of joints
robot = MobileManipulator(d, theta, a, alpha, revolute) # Create the robot model
damping_factor = 0.1

# Task definition
joint_limit_1 = np.array([-0.5, 0.5])
W = np.diag([10.0, 10.0, 1.0, 1.0, 1.0]) # Case 1: 1.0, 1.0, 10.0, 0.5, 0.5 # Case 2: 1.0, 1.0, 10.0, 10.0, 10.0 # Case 3: 10.0, 10.0, 1.0, 1.0, 1.0

tasks = [
          Configuration2D("Configuration", np.array([1.0, 0.5, np.pi/2]).reshape(3,1), 5),
        ]

# Record the data for plotting
error_end_effector = []
error_end_effector_orientation = []
joint_record = {}

# Simulation params
dt = 1.0/60.0

# Drawing preparation
fig = plt.figure()
ax = fig.add_subplot(111, autoscale_on=False, xlim=(-2, 2), ylim=(-2,2))
ax.set_title('Simulation')
ax.set_aspect('equal')
ax.grid()
ax.set_xlabel('x[m]')
ax.set_ylabel('y[m]')
rectangle = patch.Rectangle((-0.25, -0.15), 0.5, 0.3, color='blue', alpha=0.3)
veh = ax.add_patch(rectangle)
line, = ax.plot([], [], 'o-', lw=2) # Robot structure
path, = ax.plot([], [], 'c-', lw=1) # End-effector path
point, = ax.plot([], [], 'rx') # Target
PPx = []
```

```
PPy = []

# Simulation initialization
def init():
    global tasks
    line.set_data([], [])
    path.set_data([], [])
    point.set_data([], [])
    angle = np.array(np.pi - np.random.rand()*2*np.pi).reshape(1,1)
    desired_position = np.random.rand(2,1)*3-1.5
    tasks[0].setDesired(np.vstack([desired_position, angle])) # Random target
    return line, path, point

# Simulation loop
def simulate(t):
    global tasks
    global robot
    global PPx, PPy

    ### Recursive Task-Priority algorithm
    # Initialize null-space
    P = np.identity(robot.dof)
    # Initialize output vector (joint velocity)
    dq = np.zeros((robot.dof, 1))
    # Loop over tasks
    for i in range(len(tasks)):

        tasks[i].update(robot)

        if tasks[i].isActivate() != 0:
            # Update task state
            # Padding Jacobian to match the dimension of the robot dof and compute the augmented Jacobian, then compute the DLS solution and accumulate velocity, and finally update the null-space projector
            Jacobian_padded = np.pad(tasks[i].getJacobian(), ((0, 0), (0, robot.dof - tasks[i].getJacobian().shape[1])))
            # Compute augmented Jacobian
            J_bar = Jacobian_padded @ P
            # Task definition
            x_dot = tasks[i].getFeedforwardVelocity() + tasks[i].getGainMatrixK() @ tasks[i].getError()
            # Compute task velocity & Accumulate velocity
            dq = dq + weighted_DLS(J_bar, damping_factor, W) @ (tasks[i].a * x_dot - Jacobian_padded @ dq)
            # Update null-space projector
            P = P - np.linalg.pinv(J_bar) @ J_bar
        else:
            dq = dq
            P = P
    ###
```

```python

    # Record data for plotting
    error_end_effector.append(np.linalg.norm(tasks[0].getError()[0:2]))
    error_end_effector_orientation.append(abs(tasks[0].getError()[2]))

    for j in range(len(dq)):
        joint_record[j] = joint_record.get(j, []) + [dq[j,0]]

    # Update robot
    robot.update(dq, dt)

    # Update drawing
    # -- Manipulator links
    PP = robot.drawing()
    line.set_data(PP[0,:], PP[1,:])
    PPx.append(PP[0,-1])
    PPy.append(PP[1,-1])
    path.set_data(PPx, PPy)
    point.set_data(tasks[0].getDesired()[0], tasks[0].getDesired()[1])
    # -- Mobile base
    eta = robot.getBasePose()
    veh.set_transform(trans.Affine2D().rotate(eta[2,0]) +
       trans.Affine2D().translate(eta[0,0], eta[1,0]) + ax.transData)

    return line, veh, path, point

# Run simulation
animation = anim.FuncAnimation(fig, simulate, np.arange(0, 10, dt),
                                interval=10, blit=True, init_func=init,
                                   repeat=True)
plt.show()

# Plotting the end-effector path and distance to the obstacles
tvec = np.arange(0, len(error_end_effector)*dt, dt)
plt.figure()
plt.plot(tvec, error_end_effector, label='e1 (end-effector position)',
   color='orange')
plt.plot(tvec, error_end_effector_orientation, label='e2 (end-effector
   orientation)', color='blue')
plt.xlabel('Time[s]')
plt.ylabel('Error[1]')
plt.title('Task-Priority control')
plt.legend()
plt.grid()
plt.show()

# Plotting each joint velocity
plt.figure()
for j in range(len(joint_record)):
    plt.plot(tvec, joint_record[j], label=f'joint {j+1} velocity')
plt.xlabel('Time[s]')
```

```
plt.ylabel('Velocity[1]')
plt.title('Task-Priority control')
plt.legend()
plt.grid()
plt.show()
```

A.5 Full code for three different mobile base kinematic strategies

```
from lab6_robotics import * # Includes numpy import
import matplotlib.pyplot as plt
import matplotlib.patches as patch
import matplotlib.animation as anim
import matplotlib.transforms as trans

# Robot model
d = np.zeros(3)                              # displacement along Z-axis
theta = np.array([0.2, 0.5, 0.2])            # rotation around Z-axis
alpha = np.zeros(3)                          # rotation around X-axis
a = np.array([0.4, 0.3, 0.2])                # displacement along X-axis
revolute = np.array([True, True, True])      # flags specifying the type of joints
robot = MobileManipulator(d, theta, a, alpha, revolute) # Create the robot model
damping_factor = 0.1

# Task definition
W = np.diag([1.0, 1.0, 1.0, 1.0, 1.0])

conf1 = np.array([1.5, 1.5, np.pi/4]).reshape(3,1)
# conf2 = np.array([1.0, 0.5, np.pi]).reshape(3,1)
conf3 = np.array([-1.5, -1.5, -np.pi/4]).reshape(3,1)
# conf4 = np.array([0.0, 1.5, np.pi/6]).reshape(3,1)
conf5 = np.array([1.5, -1.5, -3*np.pi/4]).reshape(3,1)
# conf6 = np.array([-1.0, 0.5, -np.pi/3]).reshape(3,1)
conf7 = np.array([-1.5, 1.5, 3*np.pi/4]).reshape(3,1)

desired_configuration = [conf1, conf3, conf5, conf7]

tasks = [
          Configuration2D("Configuration", np.array([1.0, 0.5, np.pi/2]).reshape(3,1), 5),
        ]

# Counter for switching the target
counter = -3 # Set -3 because the first 3 targets are used for initialization and will not be recorded in the data

# Record the data for plotting
error_end_effector = []
error_end_effector_orientation = []
robot_pose = {}

# Simulation params
dt = 1.0/60.0

# Drawing preparation
fig = plt.figure()
```

```
ax = fig.add_subplot(111, autoscale_on=False, xlim=(-2, 2), ylim=(-2,2))
ax.set_title('Simulation')
ax.set_aspect('equal')
ax.grid()
ax.set_xlabel('x[m]')
ax.set_ylabel('y[m]')
rectangle = patch.Rectangle((-0.25, -0.15), 0.5, 0.3, color='blue', alpha=0.3)
veh = ax.add_patch(rectangle)
line, = ax.plot([], [], 'o-', lw=2) # Robot structure
path, = ax.plot([], [], 'c-', lw=1) # End-effector path
point, = ax.plot([], [], 'rx') # Target
PPx = []
PPy = []

# Simulation initialization
def init():
    global tasks
    global counter
    line.set_data([], [])
    path.set_data([], [])
    point.set_data([], [])
    if counter < len(desired_configuration):
        tasks[0].setDesired(desired_configuration[counter]) # Set target
        counter += 1
    return line, path, point

# Simulation loop
def simulate(t):
    global counter
    global tasks
    global robot
    global PPx, PPy

    ### Recursive Task-Priority algorithm
    # Initialize null-space
    P = np.identity(robot.dof)
    # Initialize output vector (joint velocity)
    dq = np.zeros((robot.dof, 1))
    # Loop over tasks
    for i in range(len(tasks)):

        tasks[i].update(robot)

        if tasks[i].isActivate() != 0:
            # Update task state
            # Padding Jacobian to match the dimension of the robot dof and compute the augmented Jacobian, then compute the DLS solution and accumulate velocity, and finally update the null-space projector
            Jacobian_padded = np.pad(tasks[i].getJacobian(), ((0, 0), (0, robot.dof - tasks[i].getJacobian().shape[1])))
```

```
            # Compute augmented Jacobian
            J_bar = Jacobian_padded @ P
            # Task definition
            x_dot = tasks[i].getFeedforwardVelocity() + tasks[i].getGainMatrixK() @ tasks[i].getError()
            # Compute task velocity & Accumulate velocity
            dq = dq + weighted_DLS(J_bar, damping_factor, W) @ (tasks[i].a * x_dot - Jacobian_padded @ dq)
            # Update null-space projector
            P = P - np.linalg.pinv(J_bar) @ J_bar
        else:
            dq = dq
            P = P
    ###

    # Record data for plotting
    error_end_effector.append(np.linalg.norm(tasks[0].getError()[0:2]))
    error_end_effector_orientation.append(abs(tasks[0].getError()[2]))

    robot_pose['x'] = robot_pose.get('x', []) + [robot.getBasePose()[0,0]]
    robot_pose['y'] = robot_pose.get('y', []) + [robot.getBasePose()[1,0]]

    # Update robot
    robot.update(dq, dt)

    # Update drawing
    # -- Manipulator links
    PP = robot.drawing()
    line.set_data(PP[0,:], PP[1,:])
    PPx.append(PP[0,-1])
    PPy.append(PP[1,-1])
    path.set_data(PPx, PPy)
    point.set_data(tasks[0].getDesired()[0], tasks[0].getDesired()[1])
    # -- Mobile base
    eta = robot.getBasePose()
    veh.set_transform(trans.Affine2D().rotate(eta[2,0]) + trans.Affine2D().translate(eta[0,0], eta[1,0]) + ax.transData)

    return line, veh, path, point

# Run simulation
animation = anim.FuncAnimation(fig, simulate, np.arange(0, 10, dt),
                               interval=10, blit=True, init_func=init, repeat=True)
plt.show()

# Plotting the end-effector path and distance to the obstacles
tvec = np.arange(0, len(error_end_effector)*dt, dt)
plt.figure()
plt.plot(tvec, error_end_effector, label='e1 (end-effector position)', color='orange')
```

```
plt.plot(tvec, error_end_effector_orientation, label='e2 (end-effector
    orientation)', color='blue')
plt.xlabel('Time[s]')
plt.ylabel('Error[1]')
plt.title('Task-Priority control (Simultaneous rotation and translation)')
plt.legend()
plt.grid()
plt.show()

# Plotting path of the mobile base and end-effector
plt.figure()
plt.plot(robot_pose['x'], robot_pose['y'], label='Mobile base path',
    color='red')
plt.plot(PPx, PPy, label='End-effector path', color='purple')
plt.xlabel('x[m]')
plt.ylabel('y[m]')
plt.title('Path of the mobile base and end-effector \n (Simultaneous rotation
    and translation)')
plt.legend()
plt.grid()
plt.show()

# Save the data record
with open('lab6_mobile_manipulator_rotate_forward.txt', 'w') as f:
    f.write('Time[s],Mobile_base_x,Mobile_base_y,
    End_effector_x,End_effector_y\n')
    for i in range(len(error_end_effector)):
        f.write(f"{tvec[i]},{robot_pose['x'][i]},{robot_pose['y'][i]},
        {PPx[i]},{PPy[i]}\n")
```

A.6 Code for plotting path of mobile base and end-effector in experiment 3

```
from lab6_robotics import * # Includes numpy import
import matplotlib.pyplot as plt
import matplotlib.patches as patch
import matplotlib.animation as anim
import matplotlib.transforms as trans

PPx_rotate_forward = []
PPy_rotate_forward = []
PPx_forward_rotate = []
PPy_forward_rotate = []
PPx_simultaneously = []
PPy_simultaneously = []
pose_x_rotate_forward = []
pose_y_rotate_forward = []
pose_x_forward_rotate = []
pose_y_forward_rotate = []
pose_x_simultaneously = []
pose_y_simultaneously = []

with open('lab6_mobile_manipulator_simultaneously.txt', 'r') as f:
    next(f) # Skip header
    for line in f:
        time, mobile_x, mobile_y, end_effector_x, end_effector_y =
            line.strip().split(',')
        PPx_simultaneously.append(float(end_effector_x))
        PPy_simultaneously.append(float(end_effector_y))
        pose_x_simultaneously.append(float(mobile_x))
        pose_y_simultaneously.append(float(mobile_y))

with open('lab6_mobile_manipulator_forward_rotate.txt', 'r') as f:
    next(f) # Skip header
    for line in f:
        time, mobile_x, mobile_y, end_effector_x, end_effector_y =
            line.strip().split(',')
        PPx_forward_rotate.append(float(end_effector_x))
        PPy_forward_rotate.append(float(end_effector_y))
        pose_x_forward_rotate.append(float(mobile_x))
        pose_y_forward_rotate.append(float(mobile_y))

with open('lab6_mobile_manipulator_rotate_forward.txt', 'r') as f:
    next(f) # Skip header
    for line in f:
        time, mobile_x, mobile_y, end_effector_x, end_effector_y =
            line.strip().split(',')
        PPx_rotate_forward.append(float(end_effector_x))
        PPy_rotate_forward.append(float(end_effector_y))
        pose_x_rotate_forward.append(float(mobile_x))
```

```
        pose_y_rotate_forward.append(float(mobile_y))

# Plotting the end-effector path and distance to the obstacles
plt.figure()
plt.plot(PPx_rotate_forward, PPy_rotate_forward, label='Rotate-then-move', color='orange')
plt.plot(PPx_forward_rotate, PPy_forward_rotate, label='Move-then-rotate', color='blue')
plt.plot(PPx_simultaneously, PPy_simultaneously, label='Simultaneous', color='green')
plt.xlabel('x[m]')
plt.ylabel('y[m]')
plt.title('End-effector path')
plt.legend()
plt.grid()
plt.show()

# Plotting path of the mobile base and end-effector
plt.figure()
plt.plot(pose_x_rotate_forward, pose_y_rotate_forward, label='Rotate-then-move', color='orange')
plt.plot(pose_x_forward_rotate, pose_y_forward_rotate, label='Move-then-rotate', color='blue')
plt.plot(pose_x_simultaneously, pose_y_simultaneously, label='Simultaneous', color='green')
plt.xlabel('x[m]')
plt.ylabel('y[m]')
plt.title('Path of the mobile base')
plt.legend()
plt.grid()
plt.show()
```